PROJECT CHIMERA

GenAI-Powered Adaptive Honeypot System

SE23UARI131, SE23UARI074, SE23UARI136, SE23UCSE144, SE23UARI040, SE23UARI104

Problem

Traditional honeypots (Cowrie, Dionaea, T-Pot) rely on static, scripted responses and hardcoded filesystems. A sophisticated attacker fingerprints them within minutes, abandoning the session before revealing useful intelligence. Security teams need a honeypot that **dynamically generates believable infrastructure** and adapts in real time to the specific attacker engaging with it.

Research Question

Can large language models generate dynamic honeypot environments indistinguishable from real infrastructure — and do they capture more attacker intelligence than static alternatives?

Proposed Solution

Project Chimera replaces static infrastructure with a generative AI pipeline. When an attacker connects, the system constructs an entire fake corporate environment from scratch — coherent employees, believable files, consistent network topology, and realistic logs — all cross-referencing each other.

System Components

File	Role
honeypot.py	SSH server — accepts any credential
llm_engine.py	LLM terminal via OpenRouter API
state_engine.py	MITRE tagging + filesystem state
narrative_engine.py	Multi-agent world builder
prompt_evolution.py	Self-rewriting rules every 5 cmds
red_team.py	Detection risk evaluator
dashboard.py	Live Streamlit analyst dashboard

Datasets Used

Dataset	Used For
Cowrie SSH Logs	Attacker cmd sequences
MITRE ATT&CK CTI	Technique tag rules
NSL-KDD	Intrusion classification
APT Notes	Attacker profiling
SecLists	Realistic artifact names

How It Works

When an attacker SSHs into port 2222, four events happen simultaneously:

- **World generation** — narrative_engine.py invents a company: employees, hostnames, git history, cron jobs, internal emails
- **LLM terminal** — llm_engine.py feeds every command to GPT-4o-mini with the world context, returning realistic terminal output
- **MITRE tagging** — state_engine.py maps each command to ATT&CK; techniques in real time
- **Prompt evolution** — every 5 commands, prompt_evolution.py rewrites the system prompt to match the attacker's focus
- **Red team check** — every 10 commands, red_team.py evaluates whether the deception is detectable and reports risk level

Novelty vs Existing Work

Existing work (Sladic et al. 2023, Hilt 2024) places a generic LLM behind SSH with no state, no world coherence, and no adaptive behavior. Chimera contributes:

- Cross-referenced multi-agent world — employees appear in both /etc/passwd and auth.log
- In-session prompt self-modification — first honeypot whose rules evolve mid-session
- Automated red-team loop — system evaluates its own detection risk
- Canary token pipeline — fake credentials traceable if used externally

Preliminary Results

Metric	Observed
MITRE techniques/session	6 – 12
Canary hit rate	75% (3/4 profiles)
Avg dwell time (APT)	11m 20s
Prompt evolutions/session	2 – 4
Red team detection risk	Low – Medium

Next Steps

- Compare dwell time vs Cowrie baseline (2-week experiment)
- Fine-tune Phi-3 on Cowrie logs using college GPU cluster

Run: python honeypot.py | streamlit run dashboard.py | ssh admin@localhost -p 2222

Initial Model Running — Preliminary Results

Model Status

> The system is operational. The honeypot accepts live SSH connections, the LLM-backed terminal responds in real time, MITRE tags fire automatically, and the dashboard displays all sessions live.

Current working components confirmed running:

- SSH honeypot server (honeypot.py) — accepts connections on port 2222
- LLM terminal simulator (llm_engine.py) — GPT-4o-mini via OpenRouter
- Real-time MITRE ATT&CK tagger (state_engine.py) — 20 technique rules
- Live analyst dashboard (dashboard.py) — Streamlit, auto-refreshes every 2s
- Prompt evolution engine (prompt_evolution.py) — rewrites rules every 5 commands
- Narrative engine (narrative_engine.py) — generates coherent company world per session
- Red team agent (red_team.py) — evaluates detection risk every 10 commands

Preliminary Results*

*all of the testing was done in a controlled environment by simulated agents

The following results were collected from synthetic attacker sessions run against the live system. Four attacker profiles were simulated — each running a scripted command sequence characteristic of that attacker type.

Session Summary Table

Attacker Profile	Commands	Dwell Time	MITRE Techs	Canary Hit	Privilege
Script Kiddie	14	38s	6	Yes	root
Intermediate	23	3m 42s	9	Yes	root
APT	31	11m 20s	12	Yes	root
Botnet	9	12s	3	No	user

MITRE ATT&CK Techniques Observed

Across all sessions, the following MITRE ATT&CK techniques were detected:

- T1033 — System Owner Discovery (whoami, id, who)
- T1003 — Credential Dumping (cat /etc/passwd, cat /etc/shadow)
- T1548 — Privilege Escalation (sudo -l, sudo su, CVE exploit)
- T1046 — Network Service Scanning (nmap 10.0.0.0/24)
- T1552 — Unsecured Credentials (cat ~/.ssh/id_rsa, cat config.php)
- T1053 — Scheduled Task/Job (crontab -l, cat /etc/crontab)
- T1105 — Ingress Tool Transfer (wget, curl to external IPs)
- T1082 — System Information Discovery (uname -a, cat /etc/os-release)
- T1057 — Process Discovery (ps aux, top)
- T1005 — Data from Local System (cat /opt/app/config.json)

Key Findings*

*all of the testing was done in a controlled environment by simulated agents

Finding	Detail
World coherence	Employee names generated for /etc/passwd appear in auth.log and bash histories — cross-agent consistency confirmed
Canary effectiveness	3 out of 4 attacker profiles accessed planted credential files (config.php, config.json) within 5 minutes

Finding	Detail
Prompt evolution	After 5 commands, system correctly identified credential_hunting focus in 2/4 sessions and injected relevant bait files
Dialogue engine	APT profile triggered mail command, dialogue engine generated realistic 3-turn sysadmin conversation
Kill chain depth	APT profile reached Installation stage (5/7 kill chain stages) — deepest engagement observed
Response latency	Cached commands (ls, whoami, id) respond in <50ms. Novel LLM commands average 380ms via OpenRouter

DATA :

https://github.com/vnvarma39/Chimera/tree/main/data